

teleop_retargert 노드 가이드

📖 teleop 문서 진입점: [README.md](#) — 하려는 일부터 문서를 고른다.

리더 로봇의 EE pose 를 팔로워가 **실행 가능한 목표 pose 로 변환** 하는 노드다.
leader-follower 미러링 체인의 한가운데에 있으며, 두 로봇의 좌표계·작업공간·자세 관례가 달라도 동작이 전달되게 만든다.

- 소스: `src/rdfp/rdfp/teleop/teleop_retargert_node.py`
- 수학 헬퍼: `retarget_math.py`
- 실행: `ros2 run rdfp teleop_retargert` (보통은 `teleop_mirror.launch.py` 가 기동)

1. 왜 retargeting 이 필요한가

1-1. 관절을 그대로 복사할 수 없다

리더와 팔로워는 **관절 수·링크 길이·구조가 다르다**. OMY-L100 과 Panda 는 관절 개수부터 다르므로 `joint_states` 를 복사하는 joint-space 미러링은 원리적으로 성립하지 않는다.

두 로봇의 **공통 언어**는 **task-space** 다. EE pose 는 어느 로봇이든 6-DOF SE(3) 로 표현되므로 이 수준에서만 1:1 대응이 가능하다.

"리더의 관절값" 이 아니라 "리더의 손끝 움직임" 을 따라 한다.

6-DOF(Degree of Freedom)
- 위치 3개: x, y, z
- 자세 3개: roll, pitch, yaw

SE(3): Special Euclidean group in 3D:
회전 $R \in SO(3)$ 과 평행이동 $t \in \mathbb{R}^3$ 의 조합을 하나의 수학적 대상으로 묶은 것

1-2. 그런데 EE pose 를 그대로 넘겨도 안 된다

task-space 로 옮겨도 **절대 pose** 를 그대로 명령하면 세 가지가 깨진다.

문제	구체적 상황
원점이 다르다	리더 base 기준 (0.3, 0, 0.4) 가 팔로워 작업공간 안이라는 보장이 없다. 첫 명령에서 팔이 급격히 튕다
방향 관례가 다르다	리더의 +x 가 팔로워의 +x 와 같은 방향이 아닐 수 있다. 앞으로 밀었는데 옆으로 간다

문제	구체적 상황
작업공간 크기가 다르다	리더가 도달범위 40 cm, 팔로워가 80 cm 먼 리더를 끝까지 움직여도 팔로워는 절반만 쓴다 (또는 그 반대로 도달 불가)

1-3. 해결 — 클러치 앵커 기반 상대 매핑

절대 위치 대신 "클러치를 잡은 순간부터의 변위" 만 전달한다.

engage 순간 저장: L_o = 리더 EE pose, F_o = 팔로워 EE pose

이후 매 표본:

$$\Delta p = R_{align} \cdot (p_L - p_{L_o})$$

리더 변위를 팔로워 좌표계로 회전 정렬

$$p_{target} = p_{F_o} + s \cdot \Delta p$$

s = 위치 스케일

$$\Delta R = R_{align} \cdot (R_L \cdot R_{L_o}^{-1}) \cdot R_{align}^{-1}$$

$$R_{target} = \Delta R \cdot R_{F_o}$$

이 한 식이 §1-2 의 세 문제를 동시에 푼다.

문제	해결 요소
원점 불일치	F_o 기준 상대 변위 — engage 시점이 곧 원점 정렬
방향 불일치	R_{align} (roll/pitch/yaw 파라미터)
크기 불일치	s (position_scale) + 클러치 재잡기

이 식의 출력은 팔로워 base frame 기준이다. p_{F_o} 가 팔로워 EE pose 이고 R_{align} 으로 축 정렬까지 끝난 값이므로, 이후 단계(clamp·필터·발행)는 모두 팔로워 좌표계에서 이루어진다. 리더 좌표계는 $(p_L - p_{L_o})$ 변위를 뺄 때만 쓰인다.

마우스와 같은 조작감 이다. 마우스를 들었다 놓으면 커서는 그대로이고 손만 편한 위치로 돌아온다. 리더를 끝까지 움직였으면 클러치를 풀고 리더를 되돌린 뒤 다시 잡으면 된다 — 작업공간 크기 차이의 실질적 해법이다.

2. 인터페이스

/leader/ee_pose

(PoseStamped, 리더 base frame)

/ee_pose

(PoseStamped, 팔로워 base frame)

/follower/target_pose

(PoseStamped, 팔로워 base frame)

~ /clutch

(std_srvs/SetBool)

engage / disengage

~ /clutch_state

(rdfp_msgs/ClutchState)

상태 변경 발행 (TRANSIENT_LOCAL)

~ /get_clutch_state

(std_srvs/Trigger)

동기 조회 (읽기 전용)

방향	이름	타입	역할
구독	leader_pose_topic (기본 leader/ee_pose)	PoseStamped	리더 EE pose. 매핑과 발행의 트리거
구독	follower_pose_topic (기본 ee_pose)	PoseStamped	팔로워 현재 EE pose. engage 앵커 F ₀ 용
발행	target_pose_topic (기본 follower/target_pose)	PoseStamped	목표 pose. frame_id 는 follower_base_frame
서비스	~/clutch	std_srvs/SetBool	true =engage, false =disengage
발행	~/clutch_state	rdfp_msgs/ClutchState	상태 변경 시에만 발행. TRANSIENT_LOCAL
서비스	~/get_clutch_state	std_srvs/Trigger	동기 조회. 상태를 바꾸지 않는다

```
ros2 service call /teleop_retarget/clutch std_srvs/srv/SetBool "{data: true}"
# → success=True, message='engaged'
```

상태 확인 — 토픽과 서비스 두 가지

~/clutch 는 SetBool 이라 호출 자체가 상태를 바꾼다. disengaged 일 때 {data: true} 를 부르면 engage 되어 버리므로 조회에 쓸 수 없다. 그래서 상태 확인 수단을 따로 둔다.

수단	방식	쓸 때
~/clutch_state 토픽	push	자동 해제를 풀링 없이 즉시 감지. GUI·상위 제어 로직
~/get_clutch_state 서비스	pull	스크립트에서 한 번 물어볼 때

토픽 — ~/clutch_state (rdfp_msgs/ClutchState)

상태가 바뀔 때만 발행한다(기동 시 초기값 1회 포함). QoS 가 TRANSIENT_LOCAL 이라 **늦게 구독해도 현재 상태를 즉시 받는다** — 이 프로젝트의 /session 토픽과 같은 규약이다.

```
ros2 topic echo /teleop_retarget/clutch_state --once      # 현재 상태
ros2 topic echo /teleop_retarget/clutch_state            # 변화 감시
```

```
header: {stamp: {...}, frame_id: ''}      # 상태가 바뀐 시각
engaged: false
reason: leader pose stream stale for 0.60s
```

서비스 — ~/get_clutch_state (std_srvs/Trigger)

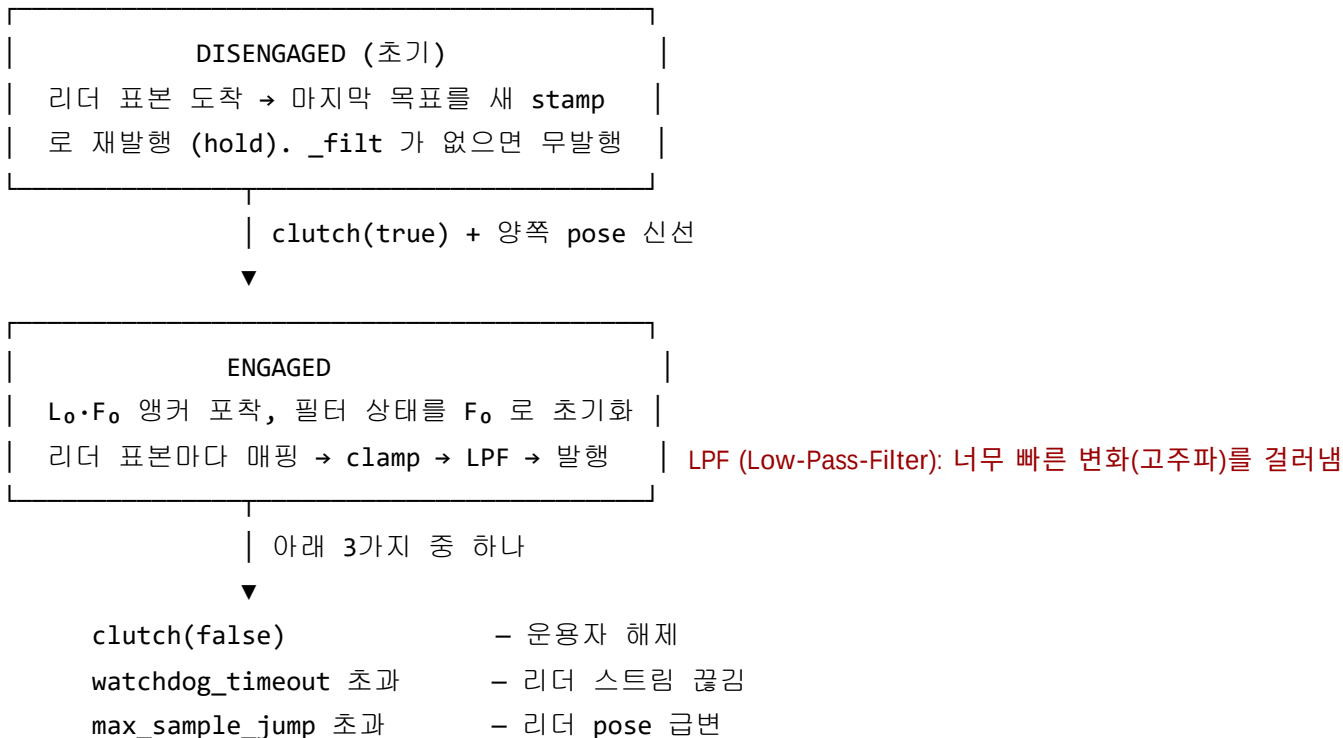
```
ros2 service call /teleop_retarget/get_clutch_state std_srvs/srv/Trigger
```

success 가 곧 engaged 여부이고, message 에는 **마지막 해제 사유** 가 담긴다 — 로그를 뒤지지 않고도 왜 풀렸는지 알 수 있다.

상황	응답
기동 후 아직 engage 전	success=False, message='disengaged: not engaged since startup'
engaged	success=True, message='engaged'
운용자 해제	success=False, message='disengaged: clutch released by operator'
watchdog 자동 해제	success=False, message='disengaged: leader pose stream stale for 0.50s'
점프 자동 해제	success=False, message='disengaged: leader pose jumped 0.312m > 0.200m'

발행은 **리더 표본이 올 때마다** 일어난다. 자체 타이머 발행이 없으므로 리더 스트림이 끊기면 출력도 멈춘다.

3. 클러치 상태 기계



3-1. engage 조건

두 조건을 **모두** 만족해야 한다. 하나라도 실패하면 서비스가 `success=False` 와 사유를 반환한다.

```
now - 리더 pose 수신시각 <= pose_staleness (기본 1.0s)
now - 팔로워 pose 수신시각 <= pose_staleness
```

```
cannot engage: leader pose is missing or stale
cannot engage: follower pose is missing or stale
```

판정은 **수신 시각** 기준이지 **header.stamp** 기준이 아니다. 따라서 오래된 stamp 를 가진 bag 재생으로도 engage 자체는 성공한다.

engage 기준이 watchdog 보다 느슨하다.

시점	파라미터	기본값
engage	pose_staleness	1.0초
engage 후 감시	watchdog_timeout	0.5초

engage 는 "최근 1초 안에 메시지가 있었는가" 만 보므로, **2 Hz 미만의 느린 스트림도 engage 는 성공하고 곧바로 watchdog 에 걸려 해제된다.** 서비스 응답만으로 스트림 건전성을 판단하지 말고 주기를 함께 확인한다.

```
timeout 3 ros2 topic hz /leader/ee_pose
```

engage 순간 필터 상태를 f_0 로 초기화하므로 **목표가 팔로워 현재 위치에서 연속적으로 출발한다.** engage 직후 튜는 현상이 없다.

3-2. disengage 시 동작 — 발행을 멈추지 않는다

해제되어도 **마지막 목표 pose 를 새 stamp 로 계속 재발행 한다(hold).**

이유는 하류 때문이다. ee_twist_publisher 가 pose 를 차분하므로, 같은 위치가 반복되면 **zero twist** 가 나와 팔로워가 현재 위치를 능동적으로 유지한다.

발행을 끊으면 moveit_servo 가 incoming_command_timeout (0.1s) 초과로 halt 하며 감속 명령을 뿌리는데, 그보다 짧끔하다.

운영상 함정 — 유량은 그대로인데 값만 안 변하므로 **동작 중인 것처럼 보인다.**

"토픽은 흐르는데 팔이 안 움직인다" 의 가장 흔한 원인이다 (§7-1).

3-3. 자동 해제 3종

조건	파라미터	목적
리더 스트림 끊김	watchdog_timeout (0.5s)	리더 통신 두절. 0.1s 주기 타이머가 감시
리더 pose 급변	max_sample_jump (0.2 m)	TF 클리치·추적 이상 시 팔로워 급발진 방지
페달 하트비트 끊김	pedal_timeout (0 = 비활성)	풋페달 데드맨

```
Clutch disengaged: leader pose stream stale for 0.55s
Clutch disengaged: leader pose jumped 0.312m > 0.200m
Clutch disengaged: pedal heartbeat stale for 0.31s
Clutch disengaged: pedal heartbeat never received
```

페달 하트비트 (pedal_timeout)

hold-to-engage 풋페달을 **진짜 데드맨으로** 만드는 장치다. 페달 노드가 밟고 있는 동안 ~/pedal_heartbeat (std_msgs/Empty) 를 주기적으로 보내고, 그것이 끊기면 자동 해제한다.

페달 노드가 죽거나 USB 가 빠지면 disengage 를 보낼 주체가 사라진다 — 종료 혹은 정상 종료만 커버하므로, 크래시·SIGKILL·USB 분리는 이 하트비트만이 막는다.

```
ros2 run rdfp teleop_retarget --ros-args -p pedal_timeout:=0.3
```

teleop_mirror.launch.py 로 띄우면 **런치가 값을 자동 보정한다** — 페달을 켜면 (enable_clutch_pedal:=true , hold) 지정하지 않아도 0.3 이 들어가고, toggle 모드면 0 으로 되돌린다. 두 조각(페달 노드 + 하트비트 감시)이 어긋난 채로 뜨는 것을 막기 위함이다. 규칙은 [clutch_pedal_guide.md §4-1](#).

pedal_timeout > 0 이면 **하트비트 없이 engage 한 경우도 즉시 해제된다** (pedal heartbeat never received). 페달 없이 콘솔·GUI 로만 조작하려면 기본값 0 을 유지한다.

현재 적용값은 이렇게 확인한다.

```
ros2 param get /teleop_retarget pedal_timeout      # Double value is: 0.3
ros2 topic info /teleop_retarget/pedal_heartbeat  # Subscription count: 1 이면 활성화
```

둘 다 자동 재engage 하지 않는다. 운용자가 상황을 확인한 뒤 다시 engage 해야 한다 — 안전을 위한 의도적 설계다.

4. 처리 순서

engaged 상태에서 리더 표본 하나가 도착하면:

1. stamp 차분 dt 계산	좌표계	(무관)	dt <= 0 (중복·역행 stamp) 이면 폐기
2. 점프 검사	리더	max_sample_jump	초과 → disengage 후 반환
3. compute_target()	리더→팔로워	앵커 기준 상대 매핑 (§1-3 수식)	
4. clamp_to_box()	팔로워	workspace	박스가 설정된 경우만
5. LPF	팔로워	위치 lerp + 자세 slerp	
6. 발행	팔로워	frame_id = follower_base_frame,	stamp = 리더 stamp

3번에서 좌표계가 바뀐다. 점프 검사(2번)만 리더 좌표계에서 하고, clamp 이후는 전부 팔로워 좌표계다 — max_sample_jump 는 리더 움직임 기준, workspace_min/max 는 팔로워 도달 영역 기준이라는 뜻이다.

필터가 clamp 뒤에 온다. 박스 경계에서 목표가 튀지 않고 부드럽게 붙는다.

LPF 계수는 표본 간격에 적응한다.

```
alpha = dt / (dt + 1/(2π·cutoff))      # cutoff <= 0 이면 alpha = 1.0 (필터 off)
```

5. 파라미터 레퍼런스

아래는 **노드 자체의** 파라미터와 기본값이다 (ros2 run rdfp teleop_retarget 로 직접 띄웠을 때). teleop_mirror.launch.py 로 띄우면 값이 <rdfp share>/config/teleop_mirror.yaml 에서 오고, launch argument 이름도 일부 다르다 (clutch_pedal.mode → pedal_mode 등). 대응표는 [launch/README.md](#) 의 " teleop_mirror — YAML ↔ 인자 대응" 절에 있다.

5-1. 토픽 / 프레임

이름	기본값	설명
leader_pose_topic	leader/ee_pose	리더 EE pose 입력
follower_pose_topic	ee_pose	팔로워 현재 EE pose 입력 (앵커용)
target_pose_topic	follower/target_pose	목표 pose 출력
follower_base_frame	panda_link0	출력 header.frame_id

5-2. 매핑

이름	기본값	단위	설명
position_scale	1.0	배	리더 변위 → 팔로워 변위 배율. > 0 필수
align_roll	0.0	rad	R_align roll
align_pitch	0.0	rad	R_align pitch
align_yaw	0.0	rad	R_align yaw. π 면 좌우 반전(거울상)

R_align 은 rpy 순서로 합성되어 리더 base → 팔로워 base 방향을 정렬한다.

5-3. 필터 / 안전장치

이름	기본값	단위	설명
lpf_cutoff_hz	3.0	Hz	저역통과 차단주파수. ≤ 0 이면 필터 비활성
watchdog_timeout	0.5	s	리더 스트림 끊김 판정. > 0 필수
max_sample_jump	0.2	m	표본 간 위치 점프 한계. ≤ 0 이면 검사 비활성
pose_staleness	1.0	s	engage 시 pose 신선도 한계. > 0 필수
pedal_timeout	0.0	s	페달 하트비트 감시. ≤ 0 이면 비활성. 풋페달 데드맨용 (가이드)
workspace_min	(미설정)	m	목표 clamp 박스 하한 [x, y, z] . 팔로워 base frame 기준
workspace_max	(미설정)	m	목표 clamp 박스 상한 [x, y, z] . 팔로워 base frame 기준

⚠ pedal_timeout 의 0.0 은 노드 자체의 기본값이다.

teleop_mirror.launch.py 로 띄우면 런치가 enable_clutch_pedal / pedal_mode

를 보고 값을 자동 보정하므로 실제 적용값이 다를 수 있다

(hold + 미지정 → 0.3 , toggle → 0.0). 상세는

[clutch_pedal_guide.md §4-1](#). 실제 값 확인:

```
ros2 param get /teleop_retarget pedal_timeout
```

박스는 팔로워 좌표계다

clamp 는 매핑이 끝난 뒤 적용되므로 대상은 follower_base_frame (기본 panda_link0) 기준 좌표다. 리더 치수와는 무관하며, 팔로워의 도달 가능 영역을 보고 값을 잡는다.

```
compute_target() → p_target = p_Fo + s·R_align·(p_L - p_Lo) ← 이미 팔로워 좌표
↓
clamp_to_box() → 이 값을 박스로 제한
```

position_scale (s)이 곱해진 뒤 잘리므로, 스케일을 키워 리더 변위를 확대해도 최종 목표는 항상 박스 안에 머문다. 안전 영역이 스케일과 독립적으로 보장된다.

```
# Panda base 기준: 몸통 앞 0.1~0.8 m, 좌우 ±0.5 m, 바닥에서 0.05 m 위
workspace_min:="[0.1, -0.5, 0.05]"
workspace_max:="[0.8, 0.5, 0.9 ]"
```

확인:

```
ros2 topic echo /follower/target_pose --once
# frame_id: panda_link0 ← 팔로워 base
# position: {x: ..., y: ..., z: ...} ← 박스 안에 있어야 한다
```

설정 규칙

workspace_min / max 는 둘 다 설정하거나 둘 다 비워야 한다. 한쪽만 주거나 길이가 3이 아니면 기동 시 ValueError 로 죽는다. 축별로 min <= max 도 검사한다.

```
workspace_min/max must both be 3-element arrays.
workspace_min must be <= workspace_max per axis.
```

두 파라미터는 Parameter.Type.DOUBLE_ARRAY 로 타입만 선언되어 있다. 빈 리스트를 기본값으로 주면 rclpy 가 BYTE_ARRAY 로 잘못 추론해 이후 launch 의 set 이 조용히 실패하기 때문이다.

6. 튜닝 가이드

좌우 반전 미러링

마주 보고 조작할 때는 리더의 좌우가 팔로워의 좌우와 반대다.

```
align_yaw:=3.14159
```

작업공간 크기 보정

리더 도달범위 40 cm, 팔로워 80 cm 라면:

```
position_scale:=2.0
```

정밀 작업에는 오히려 축소 매핑이 유용하다 (position_scale:=0.5 → 리더를 크게 움직여야 팔로워가 조금 움직임 = 미세 조작).

안전 영역 설정

position_scale 과 workspace_min/max 는 기준 좌표계가 다르다. 혼동하기 쉬운 지점이다.

파라미터	무엇을 기준으로 잡나
position_scale	리더와 팔로워의 도달범위 비율
max_sample_jump	리더 가 한 표본 사이에 움직일 수 있는 최대 거리
workspace_min / max	팔로워 base frame 의 안전 영역 (§5-3)

스케일을 키울수록 팔로워가 박스 경계에 닿는 일이 잦아지므로, position_scale 을 올릴 때는 박스도 함께 점검한다.

손떨림 제거

사람 손떨림은 8~12 Hz, 의도적 동작은 5 Hz 이하다.

lpf_cutoff_hz	특성
1.0 ~ 2.0	매우 부드럽지만 반응 지연이 체감됨

lpf_cutoff_hz	특성
3.0 (기본)	50 Hz 리더 기준 균형점
5.0 ~ 10.0	반응 빠르나 떨림이 그대로 전달
0	필터 off (디버깅용)

안전 여유

리더 통신이 불안정하면 자동 해제가 찾아진다.

```
watchdog_timeout:=1.0 max_sample_jump:=0.3
```

너무 키우지 않는다. 이 두 값은 데드맨 스위치라, 늘릴수록 이상 상황에서 팔로워가 움직이는 시간이 길어진다.

7. 문제 해결

7-1. 목표 pose 는 발행되는데 값이 안 변한다

disengaged 상태의 hold 재발행 이다 (§3-2). 로그를 확인한다.

```
# teleop_mirror 를 띄운 터미널에서
[teleop_retarget] Clutch disengaged: leader pose stream stale for 0.55s
```

다시 engage 하면 된다. 반복되면 watchdog_timeout 을 늘리거나 리더 통신을 점검한다.

7-2. engage 가 거부된다

```
cannot engage: leader pose is missing or stale
cannot engage: follower pose is missing or stale
```

해당 토픽이 실제로 흐르는지 먼저 본다.

```
ros2 topic hz /leader/ee_pose
ros2 topic hz /ee_pose          # 팔로워 - 스택이 ee_pose_node 를 띄웠는지
```

팔로워 pose 는 `rdpanda_mock / rdpanda_jgpc_mock` 이 기동하는 `ee_pose_publisher` 가 공급한다. `replay_panda_mock` 에는 **없으므로** 그 스택에서는 `retarget` 을 쓸 수 없다.

7-3. engage 직후 팔이 튈다

정상 구현에서는 발생하지 않는다 — 필터 상태를 F_0 로 초기화하기 때문이다. 그래도 튈다면 하류 게인(`twist_linear_gain`)이나 `position_scale` 이 과도한 경우다.

7-4. 방향이 엉뚱하다

`R_align` 미설정이다. 리더를 +x 방향으로 천천히 밀면서 팔로워가 어디로 가는지 보고 `align_yaw` 를 90° (1.5708) 단위로 맞춰 나간다.

7-5. 노드가 기동하자마자 죽는다

파라미터 검증 실패다. 로그의 `ValueError` 메시지를 그대로 읽으면 된다.

메시지	원인
<code>watchdog_timeout must be > 0.</code>	0 또는 음수
<code>pose_staleness must be > 0.</code>	0 또는 음수
<code>position_scale must be > 0.</code>	0 또는 음수
<code>workspace_min/max must both be 3-element arrays.</code>	한쪽만 설정 / 길이 오류

8. 단독 실행

`teleop_mirror.launch.py` 없이 노드만 띄울 때:

```
ros2 run rdfp teleop_retarget --ros-args \
  -p leader_pose_topic:=/leader/ee_pose \
  -p follower_pose_topic:=/ee_pose \
  -p target_pose_topic:=/follower/target_pose \
  -p follower_base_frame:=panda_link0 \
  -p position_scale:=1.0 \
  -p align_yaw:=0.0 \
  -p lpf_cutoff_hz:=3.0 \
  -p workspace_min:="[0.1, -0.5, 0.05]" \
  -p workspace_max:="[0.8, 0.5, 0.9]"
```

동작 확인:

```
ros2 topic hz /follower/target_pose
ros2 topic echo /follower/target_pose --once
ros2 service call /teleop_retarget/clutch std_srvs/srv/SetBool "{data: true}"
```

관련 문서

- [omy_leader_teleop_guide.md](#) — 전체 체인 기동 절차 (실행 순서·진단)
- [leader_follower_mirroring_design.md](#) — 설계 근거, 구동 방식 3가지 비교, 안전장치 설계
- [external_input_adapters.md](#) — 리더 pose 공급자가 지켜야 할 계약
- [../moveit/servo_client_programmers_guide.md](#) — 하류 servo